

大模型部署：Rerank 模型的部署及使用

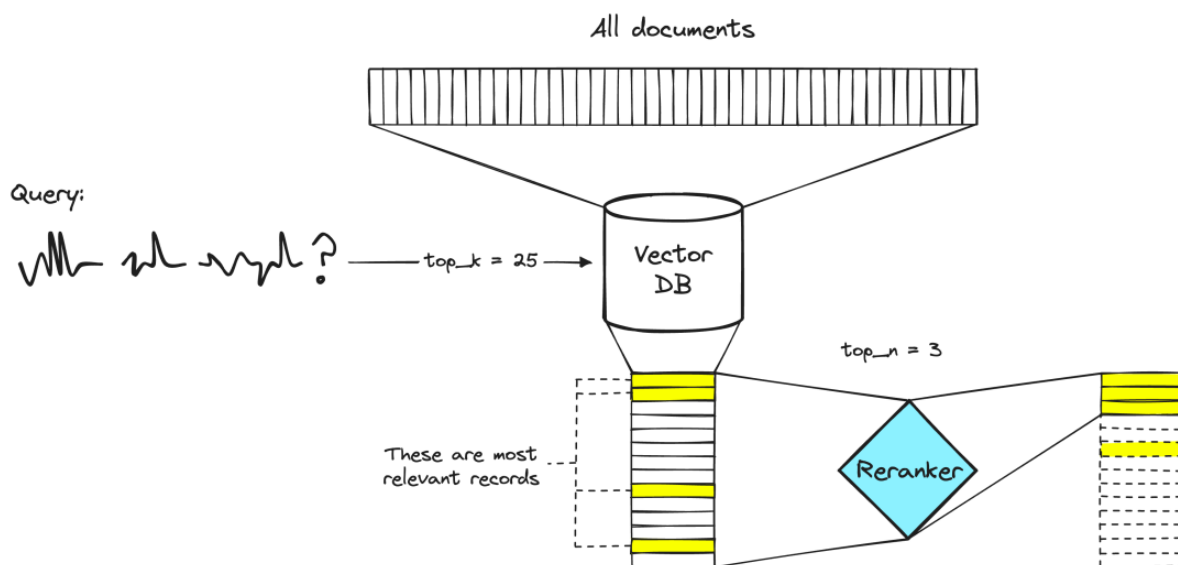
在RAG（检索增强生成）流程中，Rerank模型发挥着至关重要的作用。尽管传统的RAG能够检索出大量文档，但并非所有文档都与查询问题紧密相关。Rerank模型能够对这些文档进行再次排序和筛选，优先展示与问题更相关的文档，以此提升RAG的整体性能。

本文旨在指导如何利用HuggingFace的Text Embedding Inference（TEI）工具来部署Rerank模型，并展示如何在LlamaIndex的RAG系统中整合Rerank功能，以增强其效果。

一、Rerank 介绍

RAG 是一种结合了信息检索和文本生成的语言模型技术。简单来说，当你向大语言模型（LLM）提出一个问题时，RAG 首先会在一个大型的文档集中寻找相关信息，然后再基于这些信息生成回答。

Rerank 的工作就像是一个智能的筛选器，当 RAG 从文档集中检索到多个文档时，这些文档可能与你的问题相关度各不相同。有些文档可能非常贴切，而有些则可能只是稍微相关或者甚至是不相关的。这时，Rerank 的任务就是评估这些文档的相关性，然后对它们进行重新排序。它会把那些最有可能提供准确、相关回答的文档排在前面。这样，当 LLM 开始生成回答时，它会优先考虑这些排名靠前的、更加相关的文档，从而提高生成回答的准确性和质量。通俗来说，Rerank 就像是在图书馆里帮你从一堆书中挑出最相关的那几本，让你在寻找答案时更加高效和精准。



二、Rerank 模型部署

目前可用的 Rerank 模型并不多，有 Cohere[1] 的线上模型，通过 API 的形式进行调用。开源的模型有智源的bge-reranker-base[2]、bge-reranker-large[3]。今天我们将使用 bge-reranker-large 模型来进行部署演示。

Text Embedding Inference

我们将使用 HuggingFace 推出的 Text Embedding Inference（以下简称 TEI）工具来部署 Rerank 模型，TEI 是一个用于部署和提供开源文本嵌入和序列分类模型的工具，该工具主要是以部署 Embedding 模型为主，但是也支持 Rerank 和其他类型的模型的部署，同时它还支持部署兼容 OpenAI API 的 API 服务。

我们先进行 TEI 的安装，安装方式有 2 种，一种是通过 Docker 方式，另外一种是通过源码安装的方式，可以同时支持 GPU 和 CPU 的机器部署。

因为 Docker 安装需要有 GPU 的服务器，而一些云 GPU 服务器不方便使用 Docker，因此我们在 **Mac M1** 电脑上通过源码的方式来进行安装。

首先需要在电脑上安装 Rust，建议安装 Rust 的最新版本 1.75.0，安装命令如下：

```
curl --proto 'https' --tlsv1.2 -ssf https://sh.rustup.rs | sh
```

然后下载 TEI 的 github 仓库，并安装相关依赖，命令如下：

```
git clone https://github.com/huggingface/text-embeddings-inference.git
cd text-embeddings-inference
cargo install --path router -F candle -F metal
```

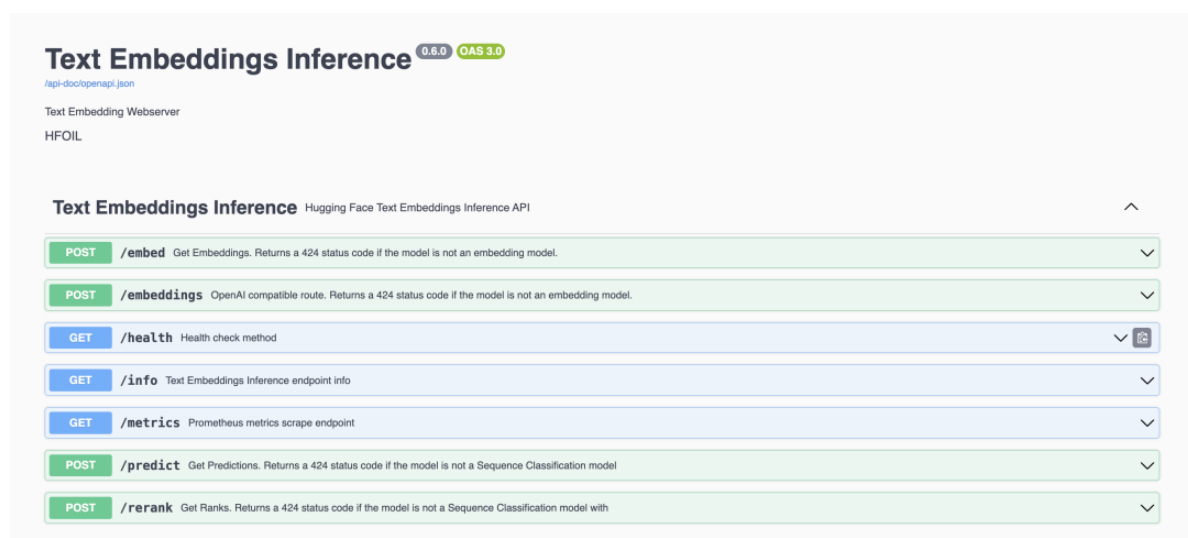
- 其中 router 是 TEI 仓库里面的一个目录
- 安装成功后，可以使用 `text-embeddings-router --help` 命令来查看工具的相关参数

TEI 安装完成后，我们使用它来部署 Rerank 模型，命令如下：

```
text-embeddings-router --model-id BAAI/bge-reranker-large --revision refs/pr/5 -
-port 8080
```

- `--model-id` 是指模型在 Huggingface 上的 ID，`revision` 是相关的版本号
- `--port` 是指服务的端口号
- 执行命令后，TEI 会从 Huggingface 上下载模型，下载到本地路径
`~/.cache/huggingface/hub/models--BAAI--bge-reranker-large`

服务启动后，我们可以在浏览器访问地址 `http://localhost:8080/docs` 来查看服务的 API 文档：



在图中可以看到有 Rerank 的接口，我们尝试用 Curl 工具来调用该接口进行验证：

```
curl -X 'POST' \
  'http://localhost:8080/rerank' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
```

```
"query": "What is Deep Learning?",
"texts": [
    "Deep Learning is ...",
    "hello"
]
}'
```

显示结果

```
[
  {
    "index":0,
    "score":0.99729556
  },
  {
    "index":1,
    "score":0.00009387641
  }
]
```

Rerank 的接口比较简单，只需要传问题 query 和相关的文档 texts 这 2 个参数即可，返回结果表示每个文档和问题的相似度分数，然后按照分数大小来进行排序，可以看到第一个文档与问题语义相近所以得分比较高，第二个文档和问题不太相关所以得分低。

需要注意的是，因为该模型是 Rerank 模型，所以如果是调用其中的 embedding 接口会报模型不支持的错误。如果你想同时拥有 Rerank 和 Embedding 的功能，可以再使用 TEI 部署一个 Embedding 模型，只要端口号不冲突就可以了。

TEI 也支持 Embedding 模型和序列分类模型的部署，其它模型的部署可以参考 TEI 的官方仓库[4]，这里就不再赘述。

三、LlamaIndex 使用 Rerank 功能

我们先来看下 LlamaIndex 普通的 RAG 功能，示例代码如下：

```
from llama_index import ServiceContext, VectorStoreIndex, SimpleDirectoryReader

documents = SimpleDirectoryReader("./data").load_data()
service_context = ServiceContext.from_defaults(llm=None)
index = VectorStoreIndex.from_documents(documents,
service_context=service_context)
query_engine = index.as_query_engine()

response = query_engine.query("健康饮食的好处是什么? ")
print(f"response: {response}")
```

- data 是放我们测试文档的目录，测试文档内容待会会介绍
- LlamaIndex 默认使用 OpenAI 的 LLM，这样的话 response 是 LLM 生成的答案，这里我们将 llm 设置为 None，response 就只会显示传递给 LLM 的提示词模板
- LlamaIndex 默认使用 OpenAI 的 Embedding 来向量化文档，因此需要设置环境变量 OPENAI_API_KEY 为你的 OpenAI API Key

- 其他部分就是 LlamaIndex 的一个普通 RAG 代码，加载目录文档，解析分块索引保存，最后进行查询

我们再来看下测试文档内容：

```
$ tree data
data/
├─ rerank-A.txt
├─ rerank-B.txt
└─ rerank-C.txt

$ cat rerank-A.txt
### 快餐的负面影响：健康与生活方式的隐忧
快餐，一种在现代快节奏生活中极为普遍的饮食选择.....

$ cat rerank-B.txt
### 选择有机，选择健康：探索有机食品的无限好处
在今天这个注重健康和可持续生活的时代.....

$ cat rerank-C.txt
### 健康饮食的益处：营养学视角的探讨
摘要：健康饮食是维持和提升整体健康的关键.....
```

这些测试文档都是和饮食相关的文档，我们执行下代码看下结果：

```
# response 显示结果
LLM is explicitly disabled. Using MockLLM.
response: Context information is below.
-----file_path: data/rerank-C.txt

### 健康饮食的益处：营养学视角的探讨
摘要：健康饮食是维持和提升整体健康的关键.....

file_path: data/rerank-A.txt

### 快餐的负面影响：健康与生活方式的隐忧
快餐，一种在现代快节奏生活中极为普遍的饮食选择.....
-----Given the context information and not prior knowledge, answer
the query.
Query: 健康饮食的好处是什么？
Answer:
```

可以看到程序会检索出和问题相似度最高的 2 个文档 `rerank-C.txt` 和 `rerank-A.txt`，但 A 文档似乎和问题关联性不大，我们可以使用 `Rerank` 来改进这一点。

我们需要使用 LlamaIndex 的 `Node PostProcessor` 组件来调用 `Rerank` 功能，`Node Postprocessor` 的作用是在查询结果传递到查询流程的下一个阶段之前，修改或增强这些结果。因此我们先来定一个自定义的 `Node PostProcessor` 来调用我们刚才部署的 `Rerank` 接口，代码如下：

```
import requests
from typing import List, Optional
from llama_index.bridge.pydantic import Field, PrivateAttr
```

```

from llama_index.postprocessor.types import BaseNodePostprocessor
from llama_index.schema import NodeWithScore, QueryBundle

class CustomRerank(BaseNodePostprocessor):
    url: str = Field(description="Rerank server url.")
    top_n: int = Field(description="Top N nodes to return.")

    def __init__(
        self,
        top_n: int,
        url: str,
    ):
        super().__init__(url=url, top_n=top_n)

    def rerank(self, query, texts):
        url = f"{self.url}/rerank"
        request_body = {"query": query, "texts": texts, "truncate": False}
        response = requests.post(url, json=request_body)
        if response.status_code != 200:
            raise RuntimeError(f"Failed to rerank documents, detail: {response}")
        return response.json()

    @classmethod
    def class_name(cls) -> str:
        return "CustomerRerank"

    def _postprocess_nodes(
        self,
        nodes: List[NodeWithScore],
        query_bundle: Optional[QueryBundle] = None,
    ) -> List[NodeWithScore]:
        if query_bundle is None:
            raise ValueError("Missing query bundle in extra info.")
        if len(nodes) == 0:
            return []

        texts = [node.text for node in nodes]
        results = self.rerank(
            query=query_bundle.query_str,
            texts=texts,
        )

        new_nodes = []
        for result in results[0 : self.top_n]:
            new_node_with_score = NodeWithScore(
                node=nodes[int(result["index"])].node,
                score=result["score"],
            )
            new_nodes.append(new_node_with_score)
        return new_nodes

```

- 我们定义了一个 `CustomRerank` 类，继承自 `BaseNodePostprocessor`，并实现了 `_postprocess_nodes` 方法

- `CustomRerank` 类有 2 个参数，`url` 是我们刚才部署的 Rerank 服务地址，`top_n` 是指返回的文档数量
- 在 `_postprocess_nodes` 方法中，我们先将原始检索到的文档转化为文本列表，再和问题一起传递给 `rerank` 方法
- `rerank` 方法会调用 Rerank 接口，这里要注意的是，TEI 中的 `texts` 参数每个文档的长度不能超过 512 个字符，如果超过了会报 413 请求参数超过限制大小的错误，这时可以将 `truncate` 参数设置为 `True`，接口会自动将过长的文档进行截断
- 得到 Rerank 的结果后，我们根据 `top_n` 参数来截取前 N 个文档，然后返回重新排序后的文档列表

我们再来看如何在 LlamaIndex 中使用 `CustomRerank`，代码如下：

```
from custom_rerank import CustomRerank

.....
query_engine = index.as_query_engine(
    node_postprocessors=[CustomRerank(url="http://localhost:8080", top_n=1)],
)

response = query_engine.query("健康饮食的好处是什么？")
print(f"response: {response}")
```

- 我们在 `as_query_engine` 方法中传递了 `node_postprocessors` 参数，这里我们将 `CustomRerank` 类传递进去
- 在 `CustomRerank` 类中，我们设置 Rerank 的 API 地址和 `top_n` 参数，这里我们设置为 1，表示只返回一个文档

修改完代码后，我们再次运行程序，可以看到结果如下：

```
# response 显示结果
LLM is explicitly disabled. Using MockLLM.
response: Context information is below.
-----file_path: data/rerank-C.txt

### 健康饮食的益处：营养学视角的探讨
摘要：健康饮食是维持和提升整体健康的关键.....

-----Given the context information and not prior knowledge, answer
the query.
Query: 健康饮食的好处是什么？
Answer:
```

可以看到这次传递给 LLM 的文档只有 `rerank-C.txt`，Rerank 只获取了最接近问题的一个文档，这样 LLM 生成的答案就更加准确了。我们可以在 `CustomRerank` 类打印原始检索的得分和经过 Rerank 后的得分，结果如下所示：

```
source node score: 0.8659382811170047
source node score: 0.8324490144594573
-----rerank node score: 0.9941347
rerank node score: 0.072374016
```

可以看到两者的得分是有差距的，这是因为原始检索和 Rerank 使用的模型不同，所以得到的分数也不同。

Rerank 模型可以帮助我们对检索到的文档进行重新排序，让相关的文档排在前面，并且过滤掉不相关的文档，从而提高 RAG 的效果。我们使用 HuggingFace 的 Text Embedding Inference 工具来部署 Rerank 模型，同时演示了如何在 LlamaIndex 的 RAG 加入 Rerank 功能。